
Rapport de déploiement

GANDAL-API (`dc-backend`)

API de gestion de VMs pédagogiques, requêtes et publications

EN PRODUCTION – OPÉRATIONNEL ET VALIDÉ

Projet	dc-backend (github.com/Delmat237/GANDAL-API)
Version	commit 1a48612 – branche <code>master</code>
Serveur cible	omega-test-3001 – 10.50.30.102
Type	Conteneurisé (Docker Compose), en isolation
Auteur	Cellule DevOps
Date	11 juin 2026

Table des matières

1	Résumé exécutif	2
2	Contexte et objectifs	2
3	Environnement cible	2
4	Architecture de la solution déployée	3
5	Procédure de déploiement réalisée	3
6	Configuration	4
6.1	Stack Docker Compose	4
6.2	Variables d'environnement (.env)	4
7	Intégration au reverse-proxy nginx	4
8	Sécurité et gestion des secrets	5
9	Recette – tests de validation	5
10	Branchement de l'hyperviseur Proxmox (intégration réelle)	6
10.1	Topologie du cluster cible	6
10.2	Génération du jeton API et configuration	6
10.3	Plage de VMID dédiée	7
10.4	Ajustements apportés au client Proxmox	7
10.5	Test de clonage réel – bout en bout	7
10.6	Activation en production	8
11	Accès à l'application	8
12	Exploitation et maintenance	8
13	Incidents rencontrés et résolutions	9
14	Risques résiduels et recommandations	9
15	Annexes	10

1 Résumé exécutif

L'API GANDAL a été déployée avec succès sur le serveur 10.50.30.102 sous forme de *stack* Docker Compose isolée, **sans aucun impact** sur les applications tierces déjà hébergées sur la machine (*stacks iwm-**, *erp-cashier-**, *verifid*).

La solution expose l'API selon deux canaux complémentaires :

1. **Reverse-proxy mutualisé** (nginx *iwm-gateway*, port 80) – accès nominal de production via le sous-domaine `gandal.10.50.30.102.nip.io` ;
2. **Port hôte direct** 8000 – accès de diagnostic.

L'ensemble de la chaîne a été vérifié de bout en bout : démarrage des conteneurs, exécution des migrations de base de données, points de santé applicatifs, routage du reverse-proxy, création du compte d'administration et authentification. **Tous les tests de recette sont au vert.**

2 Contexte et objectifs

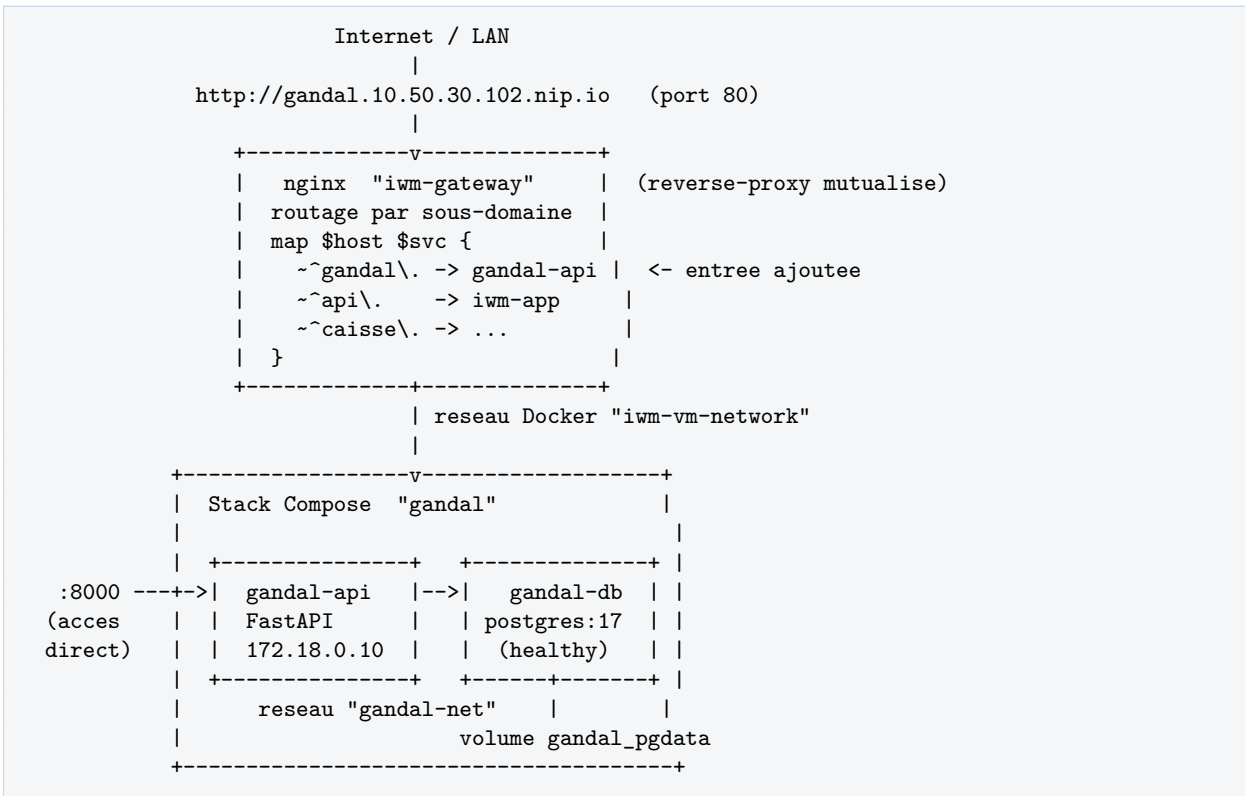
Objectif	Atteint
Déployer l'API sur <code>root@10.50.30.102</code>	OK
Ne pas perturber les services existants du serveur	OK
Exposer l'API derrière le reverse-proxy nginx existant	OK
Sécuriser les secrets de production (JWT, mot de passe admin, base)	OK
Disposer d'un compte administrateur fonctionnel	OK

3 Environnement cible

Élément	Valeur
Hôte	<code>omega-test-3001</code>
Adresse IP	<code>10.50.30.102</code>
Système	Debian GNU/Linux 12 (bookworm)
Noyau	<code>6.1.0-49-cloud-amd64</code>
Moteur de conteneurs	Docker 29.5.3
Orchestration	Docker Compose v5.1.4 (plugin)
Stockage racine	20 Go ($\approx 87\%$ utilisés après déploiement)
Répertoire applicatif	<code>/gandal-dev</code>

Note environnement partagé. Le serveur héberge plusieurs *stacks* indépendantes (*iwm-app*, *iwm-postgres*, *iwm-kafka*, *iwm-elasticsearch*, *iwm-redis*, *erp-cashier-backend*, *erp-cashier-frontend*, *verifid-backend*, *reverse-proxy iwm-gateway*). Le déploiement a été conduit en stricte isolation.

4 Architecture de la solution déployée



Inventaire des conteneurs déployés

Conteneur	Image	Taille	Réseaux	Statut
gandal-api	build local	311 Mo	gandal-net, iwm-vm-network	Up
gandal-db	postgres:17-alpine	400 Mo	gandal-net	Up (healthy)

Persistance : volume Docker nommé gandal_pgdata (données PostgreSQL).

5 Procédure de déploiement réalisée

- Audit préalable du serveur** – vérification de l'accès SSH, de la présence de Docker / Compose, des ports hôtes disponibles (80 occupé par le gateway ; 8000 et 5433 libres) et de l'absence d'un déploiement antérieur.
- Analyse du reverse-proxy existant** – identification du mécanisme de routage par sous-domaine (map \$host \$svc) et de la résolution DNS Docker sur le réseau iwm-vm-network.
- Transfert du code** – synchronisation du dépôt local vers /gandal-dev via rsync (exclusions : .git/, .venv/, caches, docs/, .env).
- Génération des secrets de production** – JWT, clé secrète SuperAdmin, mot de passe PostgreSQL et mot de passe administrateur (valeurs fortes aléatoires).
- Définition de la stack de production** – fichier docker-compose.prod.yml dédié (projet gandal, conteneurs nommés, raccordement au réseau du gateway, volume persistant, sondes de santé).
- Construction et démarrage** – docker compose up -d --build; les migrations Alembic (alembic upgrade head) s'exécutent automatiquement au démarrage du conteneur API.

7. **Intégration au reverse-proxy** – ajout d’une entrée de routage `gandal.` dans `/root/iwm/ops/nginx/gateway.conf` (avec sauvegarde préalable), validation `nginx -t` puis rechargement à chaud `nginx -s reload`.
8. **Bootstrap applicatif** – création du compte SuperAdmin via `POST /api/v1/auth/register-admin`.
9. **Recette** – validation des points de santé, du routage et de l’authentification.

6 Configuration

6.1 Stack Docker Compose

Caractéristiques notables de `/gandal-dev/docker-compose.prod.yml` :

- `name: gandal` – projet Compose isolé;
- conteneurs nommés explicitement (`gandal-api`, `gandal-db`) pour la résolution DNS par le reverse-proxy;
- `restart: unless-stopped` – redémarrage automatique des services;
- sonde de santé PostgreSQL (`pg_isready`); l’API ne démarre qu’une fois la base `healthy` (`depends_on: condition: service_healthy`);
- raccordement de `gandal-api` au réseau externe `iwm-vm-network` du gateway **et** à un réseau privé `gandal-net`;
- publication de l’API sur `0.0.0.0:8000` (accès direct de diagnostic).

6.2 Variables d’environnement (`.env`)

Variable	Valeur de production
ENVIRONMENT	production
DATABASE_URL	postgresql://dc:***@db:5432/dc_backend
JWT_SECRET	secret fort généré (<i>masqué</i>)
SUPERADMIN_USERNAME	admin
SUPERADMIN_EMAIL	admin@enspy-gi.cm
SUPERADMIN_PASSWORD	secret fort généré (<i>transmis séparément</i>)
SUPERADMIN_SECRET_KEY	secret fort généré (<i>masqué</i>)
PROXMOX_ENABLED	false (mode simulation)
EMAIL_BACKEND	log
CORS_ORIGINS	origines localhost de développement

7 Intégration au reverse-proxy nginx

Une seule ligne a été ajoutée à la table de routage du gateway mutualisé, `/root/iwm/ops/nginx/gateway.conf` :

```
map $host $svc {
    default                "";
    "~^gandal\."           "gandal-api:8000";    # <- ajout GANDAL
    "~^api-caisse\."        "erp-cashier-backend:8081";
    "~^api\."               "iwm-app:8080";
    "~^verifid\."           "verifid-backend:8080";
    "~^caisse\."            "erp-cashier-frontend:3000";
}
```

- **Sauvegarde** : une copie horodatée `gateway.conf.bak.<timestamp>` a été créée avant modification;

- **Validation** : `nginx -t` → *syntax is ok / test is successful* ;
- **Application** : rechargement à chaud (`nginx -s reload`), **sans interruption** des autres services ;
- **Non-régression** : un Host inconnu continue de renvoyer 404 `Unknown host`, confirmant que les routes existantes restent intactes.

8 Sécurité et gestion des secrets

- Tous les secrets de production ont été **régénérés** (aucune valeur d'exemple `changeme123 / change-me-*` conservée) ;
- les secrets résident dans `/gandal-dev/.env` sur le serveur ; ce fichier **n'est pas versionné** (exclu par `.gitignore` et du transfert `rsync`) ;
- le mot de passe administrateur a été communiqué **hors de ce document** ;
- la base de données PostgreSQL n'est **pas exposée** sur l'hôte (port interne au réseau Docker uniquement) ;
- le compte d'administration est protégé par une clé secrète (`SUPERADMIN_SECRET_KEY`) exigée à la création (`secrets.compare_digest`).

9 Recette – tests de validation

#	Test	Résultat attendu	Statut
1	Démarrage base (<code>gandal-db</code>)	Up (healthy)	OK
2	Migrations Alembic	révisions 0001 -> 0003	OK
3	Démarrage API	Application startup complete	OK
4	Santé – GET <code>/health</code>	<code>{"status":"ok"}</code>	OK
5	Disponibilité DB – GET <code>/ready</code>	<code>database: true</code>	OK
6	Routage gateway (Host <code>gandal.*</code>)	<code>{"status":"ok"}</code>	OK
7	Non-régression (Host inconnu)	404 <code>Unknown host</code>	OK
8	Documentation – GET <code>/docs</code>	HTTP 200 (Swagger)	OK
9	Bootstrap admin – <code>register-admin</code>	HTTP 201	OK
10	Authentification – <code>login</code>	HTTP 200 + jeton JWT	OK
11	Accès port direct <code>:8000/docs</code>	HTTP 200	OK
12	Accès via <code>nip.io /docs</code>	HTTP 200	OK

10 Branchement de l’hyperviseur Proxmox (intégration réelle)

Au-delà du *mode simulation* utilisé pour la mise en production initiale, le module d’intégration Proxmox de GANDAL a été **raccordé et validé contre un cluster Proxmox réel**. Cette section documente le branchement, les ajustements de code rendus nécessaires par le comportement réel de l’API, et le test de clonage de bout en bout.

10.1 Topologie du cluster cible

Élément	Valeur
Hyperviseur	Proxmox VE 9.2.2 (cluster à 7 nœuds)
Nœud d’API / hôte	emilia – 192.168.123.100:8006
Autres nœuds	ram (.101), rem (.102), BLADE (.103), gandal (.104), GENESIS (.105), Bris (.106)
Nœud de provisionnement	emilia (templates locaux, clonage local fiable)
Templates disponibles	2299 (omega-cloud-template, cloud-init), 9001 (omega-template-base)
Stockages	stockage.ceph (RBD partagé, 3,4 To), local-lvm (337 Go)
Pont réseau (emilia)	vibr2

Choix du nœud. Le proxy inter-nœuds *depuis* emilia renvoie une erreur 596 (Broken pipe) ; le nœud emilia a donc été retenu comme cible de provisionnement, les templates y résidant localement (clonage sans proxy). Le cluster comporte par ailleurs un nœud nommé gandal (192.168.123.104), utilisable ultérieurement en pointant directement son IP et en s’appuyant sur le stockage ceph partagé.

10.2 Génération du jeton API et configuration

Un jeton d’API dédié a été généré sur 192.168.123.100 pour l’utilisateur root@pam :

- identifiant de jeton : root@pam!gandal ;
- privsep=0 – le jeton hérite des privilèges complets (clone, configuration, démarrage, suppression) ;
- le secret n’est affiché qu’une seule fois à la création ; il est stocké dans /.env **non versionné** (exclu par .gitignore).

Variables .env correspondantes (secret masqué) :

Variable	Valeur
PROXMOX_ENABLED	true
PROXMOX_HOST	192.168.123.100
PROXMOX_USER	root@pam
PROXMOX_TOKEN_ID	gandal
PROXMOX_TOKEN_SECRET	(secret masqué – transmis séparément)
PROXMOX_DEFAULT_NODE	emilia
PROXMOX_VM_STORAGE	local-lvm
PROXMOX_NETO_TEMPLATE	virtio,bridge=vibr2
PROXMOX_VMID_RANGE_START / _END	2400 / 2499
PROXMOX_CLONE_FULL	false (clone lié)
PROXMOX_CLONE_TIMEOUT	300 (secondes)

10.3 Plage de VMID dédiée

Une plage 2400-2499 a été réservée à GANDAL et vérifiée **entièrement libre**. Elle évite toute collision avec les VMs existantes du cluster (gabarits 2299, flotte omega en 2300+ et 3000+, services en 100+). La méthode `get_next_vmid()` alloue désormais le premier identifiant libre *dans cette plage* (au lieu de `cluster/nextid` global).

10.4 Ajustements apportés au client Proxmox

Le comportement réel de l'API a révélé plusieurs écarts par rapport à l'implémentation initiale, tous corrigés dans `app/infrastructure/proxmox/proxmox_client.py` :

Ajustement	Motif / comportement réel
Attente de fin de tâche de clonage	Le clonage est asynchrone (UPID) ; la VM reste <i>verrouillée</i> (<code>lock=clone</code>) pendant la copie. Le code attend désormais la fin de la tâche avant de configurer.
Clone lié par défaut (<code>full=0</code>)	Un clone <i>complet</i> de ceph vers local-lvm (disque 20 Go) dépassait 5 minutes ; le clone lié s'effectue en quelques secondes (disque conservé sur le ceph partagé).
Clé SSH URL-encodée	Sans encodage, l'API rejette le champ (<code>invalid urlencoded string</code>). La clé est désormais encodée via <code>urllib.parse.quote</code> .
Configuration via PUT	Sur une VM fraîchement clonée, <code>POST .../config</code> plaçait les paramètres matériels (RAM, vCPU, réseau) en <i>pending</i> sans les appliquer ; le PUT synchrone les applique immédiatement.
Suppression robuste	<code>delete_vm</code> attend l'arrêt effectif de la VM (l'arrêt étant asynchrone) puis supprime avec <code>purge</code> , évitant l'erreur <code>VM is running</code> .

10.5 Test de clonage réel – bout en bout

Un cycle de vie complet a été exécuté **via le client applicatif lui-même** (lecture du `.env`, jeton réel), puis la VM de test a été détruite :

#	Étape	Résultat	Statut
1	Connexion au cluster (jeton)	Proxmox VE 9.2.2	OK
2	Allocation VMID (get_next_vmid)	2400 (dans la plage)	OK
3	Clone lié du template 2299	disque vm-2400-disk-0	OK
4	Configuration appliquée (PUT)	2048 Mo, 2 vCPU, vmbr2, clé SSH	OK
5	Démarrage (start_vm)	status=running	OK
6	Suppression (delete_vm)	VM purgée, cluster propre	OK

Durée totale clone + configuration : $\approx$ 15 secondes. L'ensemble de la suite de tests applicative (pytest) reste au vert (**102 tests**).

10.6 Activation en production

L'intégration a été développée et validée depuis le réseau de laboratoire (192.168.123.0/24). Pour l'activer sur le serveur de production (10.50.30.102), il suffit de :

1. s'assurer que l'hôte de production atteint 192.168.123.100:8006 (routage / pare-feu) ;
2. reporter les variables PROXMOX_* ci-dessus dans le .env serveur ;
3. basculer PROXMOX_ENABLED=true et PROXMOX_SIMULATION_MODE=false.

Aucune autre modification de code n'est requise.

11 Accès à l'application

Canal	URL	Usage
Reverse-proxy (nominal)	http://gandal.10.50.30.102.nip.io/	Production
Documentation	http://gandal.10.50.30.102.nip.io/docs	Production
Port hôte direct	http://10.50.30.102:8000/	Diagnostic

Résolution DNS. Le serveur s'appuie sur le service *wildcard nip.io* : toute entrée <préfixe>.10.50.30.102.nip. résout automatiquement vers 10.50.30.102. Aucune configuration DNS supplémentaire n'est donc nécessaire. En cas de bascule ultérieure sur un nom de domaine réel, il suffira de pointer gandal.<domaine> (ou un enregistrement *wildcard*) vers 10.50.30.102, sans modification côté serveur.

Compte d'administration : utilisateur admin (rôle SuperAdmin) – mot de passe transmis séparément.

12 Exploitation et maintenance

Toutes les opérations s'effectuent depuis /gandal-dev :

```
ssh root@10.50.30.102
cd /gandal-dev

# Etat des services
docker compose -f docker-compose.prod.yml ps
```

```
# Journaux applicatifs (suivi)
docker compose -f docker-compose.prod.yml logs -f api

# Mise a jour (apres synchronisation d'une nouvelle version du code)
docker compose -f docker-compose.prod.yml up -d --build

# Sauvegarde de la base de donnees
docker exec gandal-db pg_dump -U dc dc_backend > backup_$(date +%F).sql
```

Un script de déploiement automatisé est fourni : `scripts/deploy.sh` (synchronisation `rsync` + reconstruction de l'image, avec gestion des coupures réseau).

13 Incidents rencontrés et résolutions

Incident	Cause	Résolution
Coupures SSH intermittentes	Instabilité du lien réseau vers l'hôte	Encapsulation des commandes <code>ssh/scp</code> dans des boucles de réessai
Échec de <code>docker pull</code>	Résolveur DNS du serveur (10.50.30.1:53) intermittent	Réutilisation de l'image <code>postgres:17-alpine</code> déjà présente; build relancé en boucle
API non démarrée au 1 ^{er} essai	Base pas encore <code>healthy</code>	La boucle de réessai a relancé <code>up</code> ; succès au 2 ^e essai
<code>register-admin</code> rejeté (422)	Domaine e-mail <code>.local</code> réservé	Adresse valide <code>admin@enspy-gi.cm</code>
Build long interrompu	Lien réseau instable	Build exécuté détaché (<code>nohup</code>), survivant aux ruptures de session

14 Risques résiduels et recommandations

Priorité	Recommandation
Haute	Activer HTTPS/TLS sur le gateway (certificat Let's Encrypt) – le trafic transite actuellement en clair sur le port 80.
Moyenne	Restreindre ou fermer le port 8000 direct une fois la phase de diagnostic terminée.
Moyenne	Affiner <code>CORS_ORIGINS</code> avec l'origine réelle du frontend de production.
Moyenne	Sauvegarde automatisée du volume <code>gandal_pgdata</code> (cron <code>pg_dump</code> + externalisation).
Basse	Surveiller l'espace disque (87 % utilisés) – <code>docker system prune</code> régulier.
Basse	Stabiliser la résolution DNS du serveur pour fiabiliser les futurs <code>docker pull</code> .
Basse	Configurer le backend e-mail SMTP si l'envoi de notifications réelles est requis.

Priorité	Recommandation
Basse	Activer Proxmox en production : l'intégration est désormais raccordée et validée contre le cluster réel (cf. § 10) ; il reste à ouvrir la route réseau prod → hyperviseur puis à basculer <code>PROXMOX_ENABLED=true</code> .

15 Annexes

Fichiers livrés sur le serveur (/gandal-dev/)

- Code applicatif (synchronisé depuis le dépôt) ;
- `docker-compose.prod.yml` – définition de la *stack* de production ;
- `.env` – variables d'environnement et secrets (non versionné).

Fichier modifié sur le serveur

- `/root/iwm/ops/nginx/gateway.conf` – ajout de la route `gandal.` (sauvegarde `.bak.<timestamp>` conservée).

Références

- Dépôt : <https://github.com/Delmat237/GANDAL-API> ;
- Documentation API (Swagger) : `/docs` – Redoc : `/redoc` ;
- Points de santé : `/health`, `/ready`.

Rapport établi le 11 juin 2026 – déploiement réalisé et validé.